

Algoritmi e Complessità

Marco Galletti

14 aprile 2026

Indice

1	Problema 1: Sottosequenza comune più lunga (LCS)	1
2	Problema 2: Copertura di segmenti su una linea con il minimo numero di punti	3
3	Problema 3: Posizionare antenne su una strada per coprire tutte le case	5
4	Problema 4: Verificare se un valore w è pagabile con monete di taglio a, b, c	6
5	Problema 5: Ricerca del valore di maggioranza in un array (soluzione senza Boyer–Moore)	8
5.1	Soluzione con Boyer–Moore	10
6	Taglio di un intero con valore massimo	11
6.1	Taglio ottimo in tempo $O(1)$: spezzare in 3	12
7	Rilevare una sottosequenza 3-crescente in tempo lineare	14
8	Aree di sosta: massimizzare il ricavo con distanza minima 6 km	14
9	Longest Common Subsequence di tre stringhe	16

1 Problema 1: Sottosequenza comune più lunga (LCS)

Dati due stringhe s e t , di lunghezza n e m rispettivamente, vogliamo determinare una *sottosequenza comune* u di lunghezza massima tale che u sia sottosequenza sia di s sia di t .

Esempi

- **Esempio 1:** $s = \text{"abcd"}, t = \text{"aec"}$. La sottosequenza comune più lunga è "ac" di lunghezza 2.
- **Esempio 2:** $s = \text{"abc"}, t = \text{"def"}$. In questo caso non ci sono caratteri in comune nell'ordine richiesto, quindi la sottosequenza comune (LCS) è la stringa vuota, di lunghezza 0.

Soluzione tramite Programmazione Dinamica

La tecnica classica per risolvere il *Longest Common Subsequence* (LCS) è basata su programmazione dinamica.

Definizione della matrice L

Costruiamo una matrice L di dimensioni $(n + 1) \times (m + 1)$, dove $L[i, j]$ denota la lunghezza della LCS tra i primi i caratteri di s e i primi j caratteri di t . Convenzionalmente, consideriamo l'indice 0 come riferimento alla sottostringa vuota, quindi $L[0, j] = 0$ e $L[i, 0] = 0$ per ogni i, j .

Relazione di ricorrenza

Per $1 \leq i \leq n$ e $1 \leq j \leq m$:

$$L[i, j] = \begin{cases} 1 + L[i - 1, j - 1], & \text{se } s[i] = t[j], \\ \max\{L[i - 1, j], L[i, j - 1]\}, & \text{se } s[i] \neq t[j]. \end{cases}$$

Qui, $s[i]$ e $t[j]$ indicano i caratteri in posizione i di s e j di t , rispettivamente (assumendo un convenzionale 1-based indexing).

Algoritmo

Algorithm 1 LCS-LENGTH(s, t)

```
1:  $n \leftarrow \text{length}(s)$ 
2:  $m \leftarrow \text{length}(t)$ 
3: Crea una matrice  $L$  di dimensioni  $(n + 1) \times (m + 1)$ 
4: for  $i = 0 \rightarrow n$  do
5:    $L[i, 0] \leftarrow 0$ 
6: end for
7: for  $j = 0 \rightarrow m$  do
8:    $L[0, j] \leftarrow 0$ 
9: end for
10: for  $i = 1 \rightarrow n$  do
11:   for  $j = 1 \rightarrow m$  do
12:     if  $s[i] = t[j]$  then
13:        $L[i, j] \leftarrow 1 + L[i - 1, j - 1]$ 
14:     else
15:        $L[i, j] \leftarrow \max(L[i - 1, j], L[i, j - 1])$ 
16:     end if
17:   end for
18: end for
19: return  $L[n, m]$ 
```

▷ Restituisce la lunghezza della LCS

Correttezza

La correttezza dell'algoritmo di programmazione dinamica si basa sulle seguenti osservazioni:

- Se i caratteri $s[i]$ e $t[j]$ coincidono, la sottosequenza comune più lunga che termina in quella corrispondenza *aumenta* di 1 la lunghezza rispetto alla LCS dei prefissi $(i - 1)$ e $(j - 1)$.
- Se non coincidono, la LCS non può includere entrambi i caratteri (in quella posizione), e dunque prendiamo il massimo tra l'LCS escludendo $s[i]$ (cioè $L[i - 1, j]$) o quella escludendo $t[j]$ (cioè $L[i, j - 1]$).

La costruzione cumulativa di $L[i, j]$ per i da 1 a n e j da 1 a m garantisce che, quando calcoliamo $L[i, j]$, abbiamo già calcolato (quindi “memorizzato”) i valori necessari per la relazione di ricorrenza.

Analisi del tempo di esecuzione

L'algoritmo esegue un doppio ciclo **for** su $i = 1 \dots n$ e $j = 1 \dots m$, ciascuno con un corpo di operazioni in tempo $O(1)$ (un semplice confronto e due accessi a memoria). Quindi la complessità temporale è $O(n \times m)$. La memoria utilizzata è anch'essa $O(n \times m)$ per la matrice L .

Ricostruzione effettiva della LCS

Se vogliamo *non solo* la lunghezza ma anche la *sequenza* stessa, possiamo memorizzare ulteriori informazioni (o *puntatori* alle celle di provenienza) durante la costruzione di L , e alla fine risalire all'indietro partendo da $L[n, m]$. Ciò aggiunge dettagli implementativi ma non cambia la complessità asintotica.

2 Problema 2: Copertura di segmenti su una linea con il minimo numero di punti

Enunciato

Si ha un insieme di n segmenti $[l_1, r_1], [l_2, r_2], \dots, [l_n, r_n]$ su una retta numerica. Un punto p *copre* un segmento $[l_i, r_i]$ se $l_i \leq p \leq r_i$. L'obiettivo è determinare un insieme (il più piccolo possibile) di punti tale che *tutti* i segmenti siano coperti almeno da un punto.

Esempio

- Se i segmenti sono $[1, 3], [2, 5], [3, 6]$, una soluzione ottima è l'insieme di punti $\{3\}$. Infatti, il punto 3 sta in tutti e tre i segmenti.

Soluzione Greedy

Un algoritmo classico e ottimale è il seguente:

1. Ordina i segmenti in base al *loro estremo destro* in modo non decrescente. (Cioè, se abbiamo $[l_i, r_i]$ e $[l_j, r_j]$, diciamo che $[l_i, r_i]$ precede $[l_j, r_j]$ se $r_i \leq r_j$.)
2. Inizializza un insieme di punti vuoto, denotato P .
3. Itera i segmenti (già ordinati). Per il segmento corrente $[l_i, r_i]$:

- Se risulta ancora *non coperto* dai punti già scelti (cioè non esiste un punto in P che cada in $[l_i, r_i]$), **aggiungi** a P un nuovo punto posto all'estremo destro del segmento, cioè r_i .
- In questo modo, r_i copre *il segmento stesso* e potenzialmente anche qualcuno dei segmenti successivi, purché i loro intervalli inizino prima o allo stesso r_i .

Esempio di esecuzione Greedy

Consideriamo i segmenti:

$$[1, 3], \quad [2, 5], \quad [3, 6].$$

- Ordine in base all'estremo destro: $[1, 3]$, $[3, 6]$, $[2, 5]$ (in realtà tra $[3, 6]$ e $[2, 5]$ decide l'ordine chi ha estremo destro minore, quindi $[2, 5]$ precede $[3, 6]$ se guardiamo i valori: $r_2 = 5$, $r_3 = 6$. Un possibile ordine è $[1, 3]$, $[2, 5]$, $[3, 6]$.)
- Segmento $[1, 3]$: non coperto, quindi piazzo un punto $p = 3$ e lo aggiungo a P .
- Segmento $[2, 5]$: è già coperto da $p = 3$, non serve aggiungere nulla.
- Segmento $[3, 6]$: è anch'esso coperto da $p = 3$.

Risultato finale: $P = \{3\}$.

Correttezza

L'algoritmo è *corretto* e *ottimale* per un classico ragionamento greedy:

- Scegliendo il *destro* del primo segmento non ancora coperto, si copre immediatamente quel segmento e si massimizza il potenziale di coprire segmenti successivi che iniziano prima (o esattamente a) questa coordinata.
- Proseguendo fino al termine, si ottiene il minimo numero di punti.

Una dimostrazione formale spesso si fa per *contradictio*, mostrando che nessuna soluzione può usare meno punti di quelli scelti dall'algoritmo, scambiando e sostituendo se necessario.

Complessità

- L'ordinamento dei segmenti richiede $O(n \log n)$ tempo.
- La scansione finale è $O(n)$, verificando in modo efficiente se il segmento è coperto dal punto precedentemente scelto.

Quindi la complessità complessiva è $O(n \log n)$.

Conclusione

Il **greedy** che seleziona sempre l'estremo destro del primo segmento *non ancora coperto* fornisce una *soluzione ottima* in tempo $O(n \log n)$ (dovuto all'ordinamento).

3 Problema 3: Posizionare antenne su una strada per coprire tutte le case

Enunciato

Abbiamo una strada lunga t km (orientata ovest-est). Lungo questa strada ci sono k case, situate in posizioni $d_1 < d_2 < \dots < d_k$ (distanze in km dall'estremità occidentale). Vogliamo piazzare il *minor numero* di antenne lungo la strada, sapendo che *ciascuna antenna copre un raggio di 10 km* (quindi un intervallo di ampiezza totale pari a 20 km). Trovare tale numero minimo di antenne, con un esempio di posizionamento.

Esempi

- **Esempio 1:** $t = 50$, $k = 5$, e le case in posizioni $d_1 = 2$, $d_2 = 11$, $d_3 = 20$, $d_4 = 25$, $d_5 = 40$. Una soluzione ottima è 2 antenne: ad esempio, una in 12 (coprendo da 2 a 22) e una in 30 (coprendo da 20 a 40).
- **Esempio 2:** $t = 100$, $k = 6$, e le case in posizioni $d_1 = 5$, $d_2 = 15$, $d_3 = 35$, $d_4 = 45$, $d_5 = 75$, $d_6 = 85$. Una soluzione ottima è 3 antenne: ad esempio a 15 (copre 5–25), a 40 (copre 30–50), e a 80 (copre 70–90).

Soluzione Greedy

L'algoritmo segue un approccio molto simile a quello della “copertura di punti su una linea con intervalli di lunghezza fissa”. Qui, ogni antenna crea un intervallo di copertura di lunghezza 20 (da $p - 10$ a $p + 10$ se l'antenna è posta in posizione p). L'idea è:

1. Ordiniamo le case per posizione crescente. (In questo caso, abbiamo già $d_1 < d_2 < \dots < d_k$.)
2. Partiamo dalla prima casa non ancora coperta, in posizione d_i .
3. Mettiamo un'antenna a $d_i + 10$ km (cioè spostata 10 km a destra della casa).
 - In questo modo, la casa a d_i è appena coperta (è esattamente al bordo sinistro dell'intervallo di raggio 10).
 - La copertura effettiva andrà da $(d_i + 10) - 10 = d_i$ fino a $(d_i + 10) + 10 = d_i + 20$.
4. Ignoriamo tutte le case che cadono in quell'intervallo $[d_i, d_i + 20]$ (cioè tutte le case d_j con $d_j \leq d_i + 20$).
5. Se rimangono case scoperte, prendiamo la prossima casa d_j (quella più a sinistra tra le scoperte) e ripetiamo la procedura.

Commenti sul metodo

- Il posizionamento “spostato a destra di 10 km rispetto alla casa più a sinistra rimasta scoperta” è lo stesso *espedito greedy* per massimizzare la porzione coperta verso destra.
- Una volta piazzata un'antenna, tutte le case nel raggio di 20 km (fino a $d_i + 20$) sono coperte.
- Così procediamo finché non restano case scoperte.

Correttezza e ottimalità

Come per i classici problemi di *copertura su una linea*, il *ragionamento greedy* è ottimo:

- Se, per coprire la prima casa scoperta, spostassi l'antenna più a sinistra di $d_i + 10$, coprirei meno strada a destra, rischiando di dover usare più antenne.
- Se la spostassi oltre $d_i + 10$, allora non coprirebbe più la casa d_i , vanificando la scelta.

Pertanto, selezionare l'antenna a $d_i + 10$ quando si incontra la casa d_i scoperta è la strategia che minimizza il numero totale di antenne.

Analisi di complessità

- Ordinare le case (se non già ordinate) costa $O(k \log k)$.
- Il passaggio successivo (la scansione greedy) richiede $O(k)$: ad ogni passo si coprono una o più case con un'antenna, e si procede alla prossima casa scoperta.

La complessità totale è $O(k \log k)$, dominata dall'ordinamento.

Conclusione

Il numero minimo di antenne richiesto per coprire tutte le case in posizione $d_1 < d_2 < \dots < d_k$ (con raggio 10 km ciascuna) si trova con un **greedy** lineare post-ordinamento:

“Prendi la casa più a sinistra non ancora coperta, piazza l'antenna a 10 km più a destra di questa casa, salta tutte le case già coperte e ripeti.”

Questo metodo è ottimo e funziona in tempo $O(k \log k)$.

4 Problema 4: Verificare se un valore w è pagabile con monete di taglio a, b, c

Enunciato

In un paese esistono monete di tre tagli interi positivi: a , b e c . Dato un prezzo (o importo) w (anch'esso intero non negativo), vogliamo stabilire se esistono interi non negativi i_a, i_b, i_c tali che:

$$w = i_a \cdot a + i_b \cdot b + i_c \cdot c.$$

Se esiste una combinazione di monete di taglio a, b, c che somma esattamente a w , l'algoritmo deve restituire **True**; altrimenti **False**.

Esempi

- **Esempio 1:** $a = 1, b = 5, c = 7, w = 13$. Possiamo pagare $w = 13$ in diversi modi. Ad esempio, con 13 monete di valore 1, oppure 1 moneta di valore 7, 1 di valore 5 e 1 di valore 1. In tutti i casi, è possibile, quindi la risposta è **True**.
- **Esempio 2:** $a = 2, b = 4, c = 8, w = 7$. Non ci sono combinazioni non negative i_a, i_b, i_c tali che $7 = 2i_a + 4i_b + 8i_c$. Infatti, tutti i multipli di 2 e 4 e 8 sono numeri *pari*, mentre 7 è dispari; dunque **False**.

Soluzione con Programmazione Dinamica (DP)

Un metodo standard per problemi di “somme di monete” è la programmazione dinamica. Definiamo un array `dp` di dimensione `w+1`, dove:

$$\text{dp}[x] = \begin{cases} \text{True}, & \text{se è possibile pagare esattamente } x \text{ con monete di taglio } a, b, c, \\ \text{False}, & \text{altrimenti.} \end{cases}$$

L'idea è costruire `dp` in senso crescente da 0 a w .

Inizializzazione

$$\text{dp}[0] = \text{True},$$

poiché per l'importo 0 basta usare 0 monete di ciascun tipo.

Formula di transizione

Per ogni x da 1 a w :

$$\text{dp}[x] = \begin{cases} \text{dp}[x-a] \vee \text{dp}[x-b] \vee \text{dp}[x-c], & \text{se } x \geq a \text{ o } x \geq b \text{ o } x \geq c, \\ \text{False}, & \text{altrimenti.} \end{cases}$$

In termini implementativi (pseudo-codice), per ogni x :

- `dp[x] = False`
- **if** $x \geq a$ **and** `dp[x - a] == True` **then** `dp[x] = True`
- **if** $x \geq b$ **and** `dp[x - b] == True` **then** `dp[x] = True`
- **if** $x \geq c$ **and** `dp[x - c] == True` **then** `dp[x] = True`

Restituzione del risultato

Al termine, la risposta sarà `dp[w]`. Se è `True`, significa che w è pagabile con a, b, c ; se è `False`, non lo è.

Correttezza

- **Base** ($x = 0$): `dp[0] = True`, corrisponde all'uso di 0 monete.
- **Passo induttivo**: se `dp[x']` è `True`, e $x = x' + a$ (o $+b$, o $+c$) è ancora entro w , allora si può coprire x aggiungendo una moneta di valore a (o b o c). Quindi `dp[x]` diventa `True`. Viceversa, se `dp[x]` è `True`, allora x si scrive come $x' + a$, con `dp[x'] = True`.

Complessità

Il ciclo principale scorre su x da 1 a w , ciascun passo compie un numero fisso di verifiche (massimo 3). Quindi la complessità temporale è $O(w)$. Lo spazio utilizzato è $O(w)$ per l'array `dp`.

Algorithm 2 CANPAY(a, b, c, w)

```
1:  $dp \leftarrow$  array di dimensione  $w+1$ 
2:  $dp[0] \leftarrow \text{True}$ 
3: for  $x = 1$  to  $w$  do
4:    $dp[x] \leftarrow \text{False}$ 
5:   if  $x \geq a$  and  $dp[x-a] == \text{True}$  then
6:      $dp[x] \leftarrow \text{True}$ 
7:   end if
8:   if  $x \geq b$  and  $dp[x-b] == \text{True}$  then
9:      $dp[x] \leftarrow \text{True}$ 
10:  end if
11:  if  $x \geq c$  and  $dp[x-c] == \text{True}$  then
12:     $dp[x] \leftarrow \text{True}$ 
13:  end if
14: end for
15: return  $dp[w]$ 
```

Alternative

Se w fosse molto grande (e i valori di a, b, c non fornissero un vincolo sufficiente), una versione “combinatoria” del problema potrebbe essere troppo costosa in $O(w)$. In alcuni casi ci sono metodi più efficienti (ad esempio, se a, b, c sono *piccoli*, la DP funziona bene; se una combinazione di gcd-based ragionamenti e bounding limitano le ricerche, si possono esplorare altre strategie). Tuttavia, la DP a 1 dimensione rimane la soluzione standard per il caso generale con w di dimensione non troppo grande.

Conclusione

L'algoritmo di programmazione dinamica a 1 dimensione permette di stabilire in tempo $O(w)$ se esiste una combinazione non negativa di monete a, b, c che paghi esattamente l'importo w .

5 Problema 5: Ricerca del valore di maggioranza in un array (soluzione senza Boyer–Moore)

Enunciato

Sia V un array di n numeri (*floating point*). Sappiamo che *esiste* un elemento, detto valore di maggioranza, che compare in più di $n/2$ posizioni di V . Vogliamo trovare (e restituire) tale valore di maggioranza.

Soluzione tramite ordinamento

Un modo semplice (anche se non il più efficiente in senso stretto, ma comunque di complessità accettabile) è il seguente:

1. **Ordinare** l'array V . Questo richiede, in genere, tempo $O(n \log n)$.

2. **Selezionare** l'elemento $V[\lfloor n/2 \rfloor]$ (assumendo un indice 0-based) oppure $V[\lceil n/2 \rceil]$ (se 1-based).

$$candidate = V[\lceil \frac{n}{2} \rceil].$$

3. **Verificare** in un passaggio lineare se questo *candidate* compare più di $n/2$ volte.

- Se sì, lo restituiamo come valore di maggioranza.
- Altrimenti, (in un contesto generale) significherebbe che non esiste valore di maggioranza. Nel nostro caso, però, l'esistenza è garantita dall'enunciato, quindi in pratica *candidate* sarà certamente quello giusto.

Correttezza (intuizione)

Se un elemento compare in più di $n/2$ posizioni, *una volta ordinato l'array*, tale elemento occupa per forza *l'intera posizione mediana*: nella metà dell'array ci saranno più elementi uguali tra loro che qualunque altro valore. In particolare, dopo l'ordinamento, la cella di indice $\lfloor n/2 \rfloor$ (0-based) appartiene a quell'elemento.

Esempio

$$V = [10.3, 20.1, 10.3, 10.3] \quad (n = 4).$$

- Ordiniamo: $[10.3, 10.3, 10.3, 20.1]$.
- L'elemento al centro (indice $\frac{4}{2} = 2$ se 0-based, oppure posizione 2 se 1-based) è 10.3.
- Conta di 10.3: 3 occorrenze. Poiché $3 > \frac{4}{2}$, 10.3 è maggioranza.

Complessità

- L'ordinamento è $O(n \log n)$.
- La scansione di verifica è $O(n)$.

Quindi nel complesso l'algoritmo richiede $O(n \log n)$ tempo. Lo spazio dipende dal metodo di ordinamento (ad esempio $O(1)$ in-place con **Heapsort** o **Quicksort** senza considerare la ricorsione, o $O(n)$ con **Mergesort** standard).

Soluzione alternativa con conteggio (Hashing)

Qualora ci interessi un approccio lineare (a scapito di più spazio extra), possiamo usare una struttura *hash* (o un dizionario) in questo modo:

1. Inizializzare un *hash map* che, per ogni valore incontrato, mantenga un contatore di quante volte compare.
2. Scansire V e aggiornare il contatore per ciascun elemento.
3. Alla fine della scansione, l'elemento (le cui occorrenze superano $n/2$) è per forza quello di maggioranza.

Tale metodo lavora in tempo $O(n)$ *atteso* (dipende dalla *hash function* e dalle collisioni) e in spazio $O(n)$ nel caso peggiore (se quasi tutti gli elementi fossero diversi). In questo problema, però, siamo garantiti che esiste un valore che compare più di $n/2$ volte: di conseguenza, *il* valore di maggioranza occupa più della metà dell'array, e ciò si riflette in un conteggio molto elevato per quell'unico valore.

Conclusione

- **Approccio di ordinamento:** in $O(n \log n)$ troviamo il valore di maggioranza prendendo l'elemento mediano e poi verificandone il numero di occorrenze.
- **Approccio di hashing:** in $O(n)$ *atteso*, ma con $O(n)$ di spazio.

Entrambe le soluzioni trovano correttamente l'elemento di maggioranza, utilizzando concetti diversi.

5.1 Soluzione con Boyer–Moore

Enunciato

Data un'array V di n numeri in virgola mobile, sappiamo che **esiste** un valore (detto *maggioranza*) che compare in più di $\frac{n}{2}$ posizioni di V . L'obiettivo è progettare un algoritmo che individui tale valore di maggioranza in tempo lineare.

Esempi

- **Esempio 1:** se $n = 3$ e $V = [1.0, 2.0, 2.0]$, l'algoritmo deve restituire 2.0 (che appare 2 volte, $> 3/2$).
- **Esempio 2:** se $n = 4$ e $V = [10.3, 20.1, 10.3, 10.3]$, l'algoritmo deve restituire 10.3 (compare 3 volte, $> 4/2$).

Boyer–Moore Majority Vote

Un algoritmo classico per trovare un elemento di maggioranza (ammesso che esista) è il *Boyer–Moore Majority Vote*. L'algoritmo si svolge in due fasi:

Fase 1: trovare un candidato

1. Inizializza due variabili: **candidate** (non definito) e **count** = 0.
2. Scansiona l'array V da sinistra a destra. Per ciascun elemento x :
 - Se **count** = 0, assegna **candidate** = x e **count** = 1.
 - Altrimenti, se x = **candidate**, incrementa **count** di 1.
 - Altrimenti, decrementa **count** di 1.
3. Al termine, **candidate** è il candidato per la maggioranza.

Esempio rapido:

$$V = [1.0, 2.0, 2.0].$$

- Inizio: `count=0`, `candidate = undefined`.
- Leggo 1.0: `count=0`, quindi `candidate=1.0`, `count=1`.
- Leggo 2.0: `count ≠ 0`, e `2.0 ≠ 1.0`, decremento `count`, ora `count=0`.
- Leggo 2.0: `count=0`, quindi `candidate=2.0`, `count=1`.
- Fine: `candidate=2.0`.

Fase 2 (opzionale): verifica del candidato

Nel caso generale, dopo la fase 1 occorrerebbe contare quante volte `candidate` compare effettivamente in V , per verificare che sia davvero in maggioranza. Tuttavia, **se** è garantito dall'enunciato che un valore di maggioranza *esiste* di certo, *allora* il `candidate` troverà sicuramente un elemento presente più di $n/2$ volte, e non serve una verifica ulteriore.

Analisi di complessità

- **Tempo:** la Fase 1 scorre l'array una sola volta, eseguendo un numero costante di operazioni per elemento, quindi ha complessità $O(n)$. L'eventuale Fase 2 sarebbe un secondo passaggio lineare, comunque $O(n)$.
- **Spazio:** oltre all'array V , usiamo solo poche variabili (`candidate`, `count`), perciò $O(1)$ spazio extra.

Conclusione

L'algoritmo di Boyer–Moore individua il valore di maggioranza (se esiste) in tempo lineare $O(n)$ e spazio $O(1)$. Con l'ipotesi di esistenza garantita di tale valore, `candidate` al termine della scansione è proprio il valore di maggioranza.

6 Taglio di un intero con valore massimo

Enunciato. Dato un intero $n \geq 2$, un *taglio* di n è una sequenza di interi positivi $a_1, a_2, \dots, a_k$ che sommano a n ($\sum_{i=1}^k a_i = n$). Il *valore* del taglio è il prodotto $P = a_1 \cdot a_2 \cdots a_k$. Obiettivo: calcolare il valore massimo $P^*(n)$ ottenibile.

Idea. Sia $V[i]$ il valore massimo ottenibile tagliando l'intero i (potrà coincidere con i stesso, perché è lecito non suddividere). Per $i \geq 3$ consideriamo di “staccare” la parte destra di lunghezza j

$$n = i = (i - j) + j, \quad 1 \leq j \leq i - 3,$$

e moltiplicare $V[i - j] \cdot j$. Il massimo fra tutti i j e fra i (nessun taglio) è proprio $V[i]$.

Algorithm 3 MAXCUTVALUE(n)

```
1: crea array  $V[0 \dots n]$  con  $V[i] \leftarrow i$  ▷ valore senza tagli
2: for  $i \leftarrow 3$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $i - 3$  do ▷ non serve  $j = i - 2, i - 1$ 
4:      $V[i] \leftarrow \max(V[i], V[i - j] \cdot j)$ 
5:   end for
6: end for
7: return  $V[n]$ 
```

Algoritmo.

Correttezza.

Dimostrazione. Per induzione su i .

Base. $V[0] = 0$, $V[1] = 1$, $V[2] = 2$ sono corretti perché l'unico taglio ammesso è il numero stesso (o la somma di $1 + 1$ che vale $1 < 2$).

Passo induttivo. Assumiamo corretti $V[1], \dots, V[i - 1]$ e mostriamo che l'algoritmo assegna a $V[i]$ il valore ottimo.

Sia S^* un taglio ottimo di i e j la sua *ultima* parte ($j \geq 1$). Il prefisso di somma $i - j$ ha valore massimo $V[i - j]$ per l'ipotesi induttiva; quindi

$$P^*(i) = V[i - j] \cdot j.$$

Il ciclo interno considera proprio tale j e quindi in quell'iterazione assegna $V[i] \geq V[i - j] \cdot j = P^*(i)$. Poiché $V[i]$ non può eccedere il valore ottimo (ci limitiamo a tagli realizzabili), $V[i] = P^*(i)$.

Induzione completa $\Rightarrow V[n] = P^*(n)$ ed il valore restituito è corretto. $\square$

Complessità. Il doppio ciclo esegue

$$\sum_{i=3}^n (i - 3) = O(n^2)$$

operazioni elementari, quindi $T(n) = \Theta(n^2)$. Lo spazio occupato è $V[0 \dots n]$, ovvero $O(n)$.

Ricostruzione del taglio. Basta salvare, insieme a $V[i]$, l'indice j che ha massimizzato l'espressione. Risalendo da $i = n$ si ottengono tutte le parti del taglio ottimo in $O(n)$ tempo addizionale e senza aumentare l'ordine di spazio.

Osservazione sulla velocità massima. Esiste una soluzione $O(1)$ (o $O(n)$ con aritmetica *big-int*) basata sul fatto noto che il prodotto si massimizza spezzando n in quanti più 3 possibile (eccezioni: resti $= 1$ o $= 2$ gestiti con un 4 o un 2). La DP quadratica qui presentata rimane tuttavia accettata dai limiti tipici di esercizio ed è perfettamente corretta.

6.1 Taglio ottimo in tempo $O(1)$: spezzare in 3

Osservazione chiave. Per la disuguaglianza aritmetico-geometrica, a somma fissata il prodotto è massimo quando tutti i termini sono uguali; la loro lunghezza ideale sarebbe $e \simeq 2.718$, ma sui soli interi il massimo si ottiene con quante più parti di 3 possibile (eccezioni a fine sezione).

Lemma 1. Sia $n \geq 4$. Tra tutti i tagli in due parti $n = x + (n - x)$ con $x \in \{1, \dots, n - 1\}$ il prodotto $x(n - x)$ è massimizzato per $x = 2$ o $x = 3$, e vale in particolare $3(n - 3) \geq 2(n - 2) \geq x(n - x) \quad \forall x$.

Dimostrazione. La parabola $f(x) = x(n - x)$, $x \in [0, n]$, raggiunge il massimo reale in $x = \frac{n}{2}$; fra gli interi $\{1, \dots, n - 1\}$ il valore più vicino è $x = \lfloor n/2 \rfloor$. Quando $n \geq 4$ si verifica facilmente $f(3) \geq f(x)$ per ogni intero $x \neq 3$, con stretta disuguaglianza eccetto $x = 2$ per $n = 4$. $\square$

Strategia greedy globale. Procediamo dividendo n per 3:

$$n = 3q + r, \quad r \in \{0, 1, 2\}.$$

1. Se $r = 0$: il taglio ottimo è $\underbrace{3 + 3 + \dots + 3}_{q \text{ volte}}$ e il valore è 3^q .
2. Se $r = 1$: conviene sostituire *un* blocco $(3 + 1)$ con $2 + 2$, quindi $n = 3(q - 1) + 2 + 2$, valore $3^{q-1} \cdot 2^2$.
3. Se $r = 2$: basta lasciare un 2 in fondo, valore $3^q \cdot 2$.

Correttezza. Per induzione su n .

Base $n \leq 5$. Si controlla esaustivamente che la tabella seguente coincide con l'algoritmo:

n	miglior prodotto
2	2 (taglio 2)
3	3 (taglio 3)
4	4 (taglio 2 + 2)
5	6 (taglio 3 + 2)

Passo induttivo $n \geq 6$. Applichiamo il Lemma 1: qualunque taglio ottimo di n contiene almeno una parte $x \in \{2, 3\}$; rimuovendola otteniamo $n - x$, per cui (per ipotesi induttiva) il prodotto massimo residuo è $P^*(n - x)$. Confrontiamo i tre casi possibili:

$$\begin{aligned} x = 3 &\Rightarrow P^*(n) = 3 \cdot P^*(n - 3), \\ x = 2 &\Rightarrow P^*(n) = 2 \cdot P^*(n - 2), \\ x = 1 &\Rightarrow \text{mai ottimale poiché } 3P^*(n - 3) \geq 1 \cdot P^*(n - 1). \end{aligned}$$

Ora, la politica “prendi quanti più 3 possibile e gestisci il resto come sopra” produce esattamente il maggiore tra le tre espressioni ed è dunque ottima. $\square$

Complessità. Il calcolo di $q = \lfloor n/3 \rfloor$ e del resto r richiede $O(1)$ tempo ed $O(1)$ spazio. La potenza 3^q (ed eventualmente 2 o 2^2) si ottiene con un'unica istruzione `pow(3,q)` nei linguaggi con interi a precisione arbitraria (Python, Java `BigInteger`, ecc.).

Nota sugli interi molto grandi. Il numero di cifre del risultato cresce come $\Theta(n \log_{10} 3)$; pertanto qualunque algoritmo che *stampi* l'intero valore deve comunque impiegare $\Omega(n)$ tempo solo per l'output.

7 Rilevare una sottosequenza 3-crescente in tempo lineare

Problema. Dato un array $v[0 \dots n-1]$ di interi non negativi ($n \geq 3$), stabilire se esistono indici $0 \leq i < j < k \leq n-1$ tali che $v[i] < v[j] < v[k]$.

Algorithm 4 ISTHREEINCREASING

```
1:  $first \leftarrow +\infty$ 
2:  $second \leftarrow +\infty$ 
3: for  $x$  in  $v$  do
4:   if  $x \leq first$  then
5:      $first \leftarrow x$ 
6:   else if  $x \leq second$  then
7:      $second \leftarrow x$ 
8:   else
9:     return True  $\triangleright first < second < x$ 
10:  end if
11: end for
12: return False
```

Algoritmo.

Correttezza.

Dimostrazione. Mostriamo che la funzione restituisce **True** $\iff$ esiste una tripla crescente.

($\Rightarrow$) La riga 6 viene eseguita solo quando l'elemento corrente x soddisfa $x > second > first$, con $first$ e $second$ assegnati in iterazioni precedenti. Siano i l'indice che ha fissato $first$, j quello di $second$ e k l'indice corrente; per costruzione $i < j < k$ e $v[i] = first < v[j] = second < v[k] = x$.

($\Leftarrow$) Supponiamo esista una tripla di indici $i < j < k$ con $v[i] < v[j] < v[k]$. Durante la scansione:

* quando si processa $v[i]$, si pone $first = v[i]$, $second = +\infty$; * all'elemento $v[j]$ vale il ramo “**elseif**”, quindi $second \leftarrow v[j]$ e $second > first$; * all'elemento $v[k]$ abbiamo $v[k] > second$ e si attiva la riga 6, per cui l'algoritmo termina con **True**. $\square$

Analisi. La scansione visita ogni elemento una sola volta $\Rightarrow$ tempo $\Theta(n)$. Si utilizzano due variabili scalari $\Rightarrow$ spazio $O(1)$ supplementare.

8 Aree di sosta: massimizzare il ricavo con distanza minima 6 km

Enunciato

Abbiamo t aree di sosta posizionate ai chilometri $0 < x_1 < x_2 < \dots < x_t$ lungo una strada sterrata. Aprire un chiosco nell'area i produce un ricavo $e_i > 0$. Una legge vieta l'apertura simultanea di due chioschi a distanza ≤ 5 km l'uno dall'altro. Obiettivo: scegliere un sottoinsieme di aree che massimizzi il ricavo totale.

Pre-processing lineare

Per ciascuna area i definiamo

$$p(i) = \max\{j < i \mid x_i - x_j > 5\} \cup \{0\},$$

cioè l'ultima area compatibile con i (oppure 0 se non esiste). Poiché gli x_i sono in ordine crescente, possiamo calcolare tutti i $p(i)$ con due indici che scorrono in avanti in $O(t)$ tempo.

Algorithm 5 COMPUTEP

```
1: function COMPUTEP( $x[1 \dots t]$ )
2:    $p[1 \dots t] \leftarrow 0$ 
3:    $j \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $t$  do
5:     while  $j < i$  and  $x_i - x_{j+1} > 5$  do
6:        $j \leftarrow j + 1$ 
7:     end while
8:      $p[i] \leftarrow j$ 
9:   end for
10:  return  $p$ 
11: end function
```

Programmazione dinamica

$$DP[i] = \text{ricavo ottimo usando solo le prime } i \text{ aree}, \quad DP[0] = 0.$$

Ricorrenza classica dello *weighted-interval scheduling*:

$$DP[i] = \max(DP[i-1], e_i + DP[p(i)]).$$

Algorithm 6 MAXREVENUE

```
1: function MAXREVENUE( $x[1 \dots t], e[1 \dots t]$ )
2:    $p \leftarrow \text{COMPUTE}(x)$ 
3:    $DP[0 \dots t] \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $t$  do
5:      $DP[i] \leftarrow \max(DP[i-1], e_i + DP[p[i]])$ 
6:   end for
7:   return  $DP[t]$ 
8: end function
```

Correttezza

Procediamo per induzione su i .

Base $i = 0$. $DP[0] = 0$ è ottimo con zero aree.

Passo induttivo. Assumiamo ottimo $DP[i-1]$. Una soluzione ottima sulle prime i aree deve:

- **escludere** l'area i : allora il valore è $DP[i-1]$;

- **includere** l'area i : per il vincolo legale può includere solo aree con indice $\leq p(i)$, per le quali il valore ottimo è $DP[p(i)]$, quindi il totale è $e_i + DP[p(i)]$.

Prendendo il massimo dei due otteniamo un valore che non può essere migliorato, dunque $DP[i]$ è ottimo. Per induzione $DP[t]$ è il ricavo massimo globale.

Complessità

- **Tempo:** $\Theta(t)$ per COMPUTEP + $\Theta(t)$ per la DP $\Rightarrow \Theta(t)$.
- **Spazio:** due vettori di lunghezza $t + 1 \Rightarrow O(t)$.

9 Longest Common Subsequence di tre stringhe

Enunciato. Date tre stringhe

$$A = a_1 a_2 \cdots a_n, \quad B = b_1 b_2 \cdots b_m, \quad C = c_1 c_2 \cdots c_\ell,$$

vogliamo trovare una sottosequenza comune di massima lunghezza, ovvero una stringa U tale che U sia sottosequenza simultanea di A , B e C , e $|U|$ sia massimo.

Definizione del DP. Si introduce una matrice tridimensionale

$$L[i, j, k] = \text{lunghezza della LCS fra } A[1..i], B[1..j], C[1..k].$$

Per convenzione:

$$L[i, j, k] = 0 \quad \text{se almeno uno fra } i, j, k = 0.$$

Ricorrenza. Per $i \geq 1, j \geq 1, k \geq 1$,

$$L[i, j, k] = \begin{cases} 1 + L[i-1, j-1, k-1], & \text{se } a_i = b_j = c_k, \\ \max\{L[i-1, j, k], L[i, j-1, k], L[i, j, k-1]\}, & \text{altrimenti.} \end{cases}$$

Algorithm 7 LCS3-LENGTH(A, B, C)

```
1:  $n \leftarrow |A|, m \leftarrow |B|, \ell \leftarrow |C|$ 
2: crea array  $L[0..n][0..m][0..\ell]$ 
3: for  $i = 0$  to  $n$  do
4:   for  $j = 0$  to  $m$  do
5:     for  $k = 0$  to  $\ell$  do
6:        $L[i, j, k] \leftarrow 0$ 
7:     end for
8:   end for
9: end for
10: for  $i = 1$  to  $n$  do
11:   for  $j = 1$  to  $m$  do
12:     for  $k = 1$  to  $\ell$  do
13:       if  $A[i] = B[j] = C[k]$  then
14:          $L[i, j, k] \leftarrow 1 + L[i - 1, j - 1, k - 1]$ 
15:       else
16:          $L[i, j, k] \leftarrow \max(L[i - 1, j, k], L[i, j - 1, k], L[i, j, k - 1])$ 
17:       end if
18:     end for
19:   end for
20: end for
21: return  $L[n, m, \ell]$ 
```

Pseudocodice.

Correttezza.

- Se $a_i = b_j = c_k$, ogni LCS di $A[1..i], B[1..j], C[1..k]$ può terminare con quel carattere comune, aggiungendo 1 alla LCS di tre prefissi di lunghezza $(i - 1, j - 1, k - 1)$.
- Se non tutti e tre i caratteri combaciano, allora l'LCS non può includere la tripletta (i, j, k) simultaneamente: bisogna escludere almeno uno dei tre indici, e la miglior scelta equivale a prendere il massimo fra i tre casi in cui scaliamo uno soltanto dei tre indici.
- I casi base $L[*, *, 0], \dots$ sono tutti zero, corrispondenti alla sottosequenza vuota.
- L'algoritmo esplora tutti i sotto-sottoproblemi necessari e usa ciascun valore già calcolato esattamente dove occorre.

Analisi di complessità.

- Il triplo ciclo su $i = 1 \dots n, j = 1 \dots m, k = 1 \dots \ell$ esegue una costante $O(1)$ operazione per ciascuna cella.
- Vi sono $(n + 1)(m + 1)(\ell + 1)$ celle, pertanto il tempo totale è

$$T(n, m, \ell) = \Theta(n m \ell).$$

- Lo spazio occupato è anch'esso $O(n m \ell)$ per l'array L .

Ricostruzione della sottosequenza. Per ottenere la stringa effettiva (non solo la sua lunghezza), si possono memorizzare durante il calcolo dei *puntatori* (o una struttura di backtracking) in ciascuna cella, poi risalire da (n, m, ℓ) fino a uno degli stati base seguendo:

se $A[i] = B[j] = C[k]$, vai a $(i - 1, j - 1, k - 1)$ altrimenti vai nel vicino con valore max.

Il costo aggiuntivo è $O(n + m + \ell)$.